

Orpheus-TTS Hindi

GRPO Design & Mathematics

Architecture, reward mathematics, and dataset configuration for optimizing the Orpheus-TTS Hindi (3B) model with Group Relative Policy Optimization over a multi-scale SNAC neural codec.

BASE MODEL	METHOD	CODEC	HARDWARE
Orpheus-TTS Hindi · 3B params	GRPO · $G = 4$ samples/group	SNAC · 12 / 24 / 48 Hz	2×15 GB (Kaggle T4)



[shac_intercept.py](#) • [Rasa Hindi Manifest](#) • [Whisper-large-v3](#) • [ECAPA-TDNN](#) • [wav2vec2-ER](#)

Dataset & Style Ratio Strategy

The training dataset is derived from the Rasa Hindi manifest. SFT (Stage 2) was completed using normalized text, which is preserved through this RL phase.

A Mapped Speakers & Prompt Format

- **prakhar** (Male Voice) ← mapped from `gender: Male`
- **prerna** (Female Voice) ← mapped from `gender: Female`

```
User: <[speaker]> <[style]> बोलिए: "[normalized_text]"
Assistant: <|SPEECH_GENERATION_START|>
```

B Style Groups

The manifest contains multiple reading styles. To prevent the model from collapsing to a flat "neutral mean" — catastrophic forgetting of prosody — styles are categorized and sampled using an **80 / 20 ratio**.

20% · 1,000 SAMPLES

Neutral Anchor Group

`BOOK` `INDIC` `CONV` `WIKI` `NEWS` `BB` `PROPER`
`NOUN` `ALEXA` `UNKNOWN` `NORMAL`

80% · 4,000 SAMPLES

Active Emotional Group

`ANGER` `SURPRISE` and other expressive styles

PREPROCESSING NOTE

During preprocessing, stratified sampling with replacement is applied to minority emotional styles to hit the exact target sizes.

The Mathematics of GRPO

Group Relative Policy Optimization replaces standard PPO by eliminating the heavy critic (value) network, estimating baseline value directly from a group of sample completions — saving up to **50% of VRAM**.

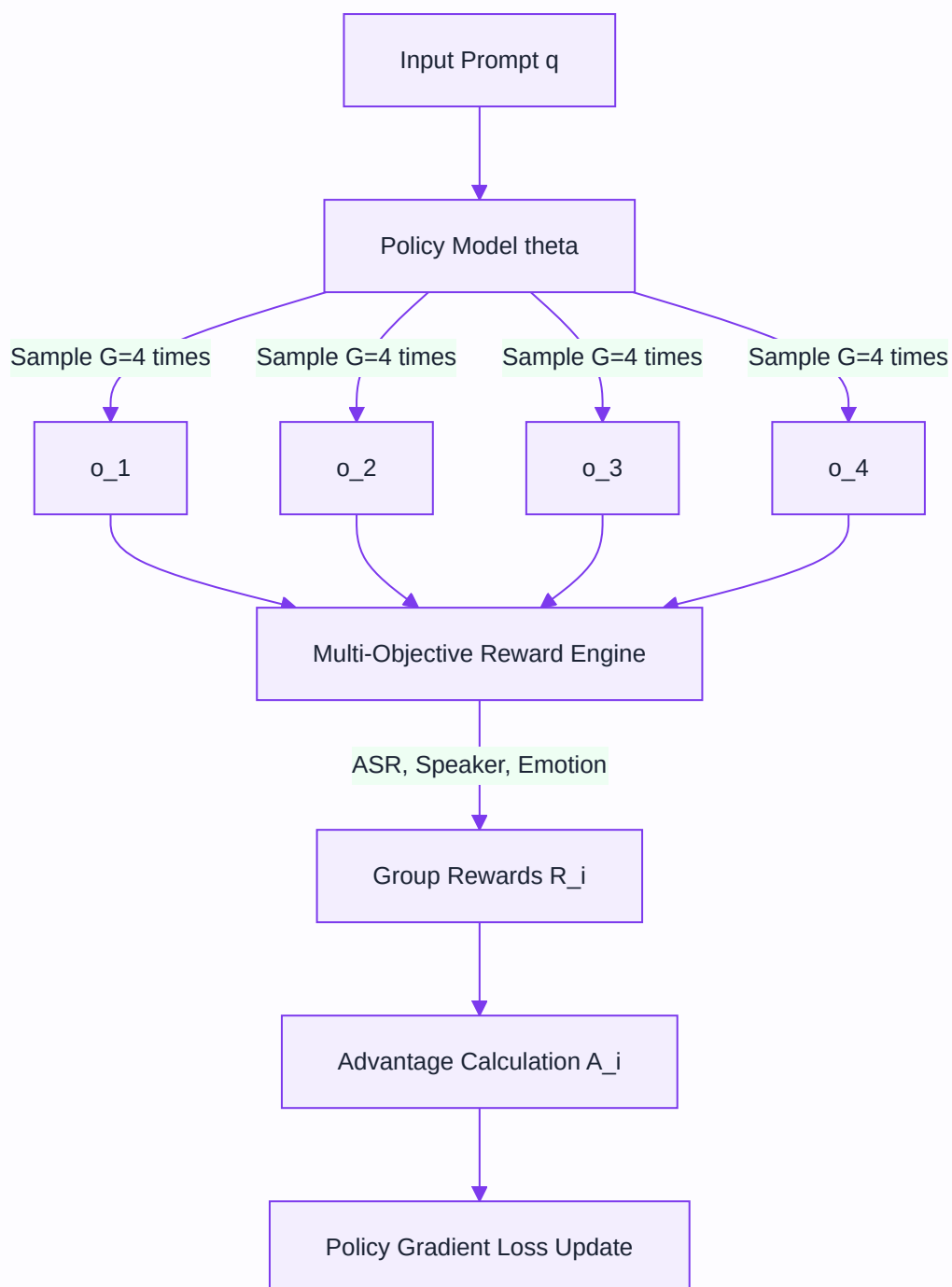


FIG. 2.1 — GROUP SAMPLING & ADVANTAGE PIPELINE

A Group Advantage Estimation

For each text prompt q sampled from the dataset, the policy model π_θ generates a group of G independent completions:

$$\{o_1, o_2, \dots, o_G\} \sim \pi_\theta(O \mid q)$$

The reward engine scores each completion, $R_i = R(q, o_i)$. The advantage A_i of completion o_i relative to the group is:

$$A_i = \frac{R_i - \text{mean}(\{R_1, \dots, R_G\})}{\text{std}(\{R_1, \dots, R_G\}) + \epsilon}$$

INTUITION

If a generation sounds better (higher reward) than the group average, $A_i > 0$ and the policy is reinforced. If it sounds worse, $A_i < 0$ and the policy is discouraged.

B GRPO Objective Function

We maximize the clipped policy objective with a KL-divergence penalty to prevent policy drift relative to the SFT reference model π_{ref} :

$$L_{\text{GRPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G [\min(r_i(\theta)A_i, \text{clip}(r_i(\theta), 1 - \epsilon, 1 + \epsilon)A_i) - \beta D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})]$$

Where:

- $r_i(\theta) = \frac{\pi_\theta(o_i \mid q)}{\pi_{\theta_{\text{old}}}(o_i \mid q)}$ — the probability ratio
- ϵ is the clipping parameter (typically 0.2)
- β is the KL penalty coefficient (typically 0.01)

$$D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}}) = \frac{\pi_{\text{ref}}(o_i \mid q)}{\pi_\theta(o_i \mid q)} - \log \frac{\pi_{\text{ref}}(o_i \mid q)}{\pi_\theta(o_i \mid q)} - 1$$

Approximated online, per-token, during rollout scoring.

SECTION 03

Custom SNAC Token Unweaving

Orpheus-TTS represents speech waveforms using the **SNAC** multi-scale neural codec. For 24 kHz audio, SNAC compresses audio into three temporal layers.

LEVEL 1
Quantizer 0 — 12 Hz
Coarse / global prosody

LEVEL 2
Quantizer 1 — 24 Hz
Medium features

LEVEL 3
Quantizer 2 — 48 Hz
Fine details

These tokens are interleaved into a single flat sequence of token IDs using a **7-token frame pattern** representing 1 frame at 12 Hz:

$$\text{Frame}_i = [Q_0(i), Q_1(2i), Q_2(4i), Q_2(4i+1), Q_1(2i+1), Q_2(4i+2), Q_2(4i+3)]$$

A Vocabulary Offset Translation

Orpheus-TTS maps these layers into different partitions of its vocabulary using a base offset of **128266** and a codebook size of **4096**.

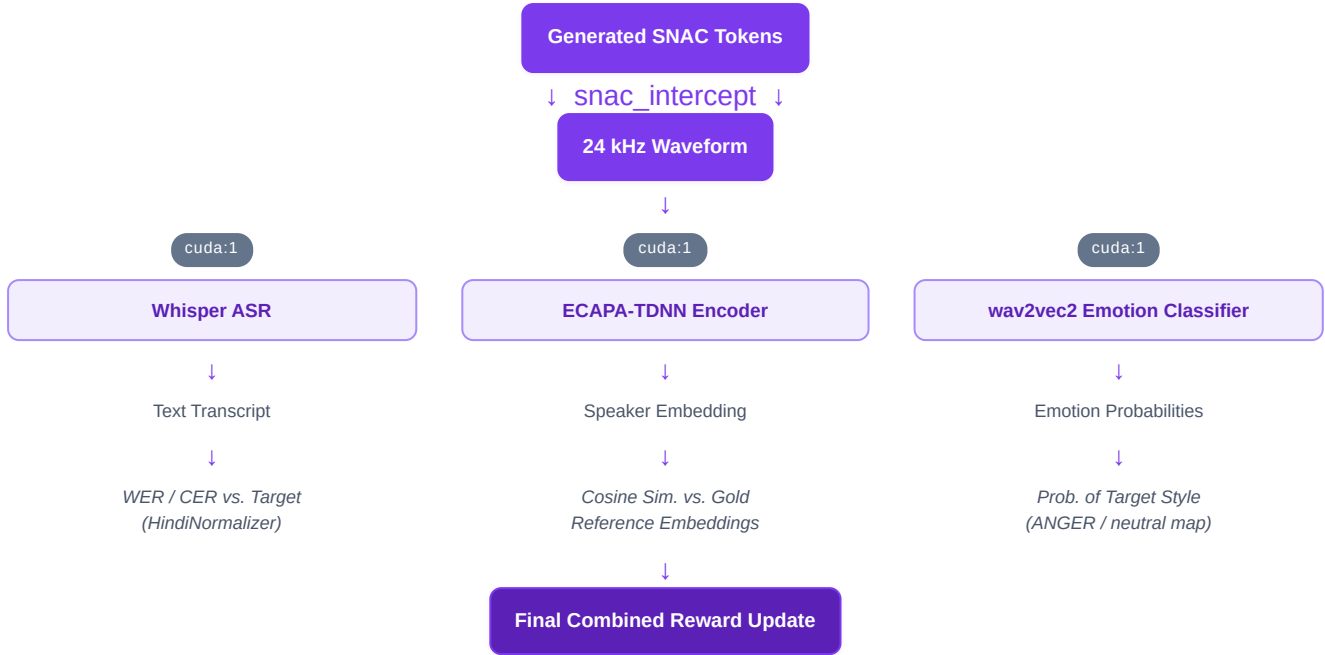
QUANTIZER INDEX	SNAC LEVEL	OFFSET RANGE IN TOKENIZER VOCABULARY
Q_0	Level 1	[128266, 132361]
Q_1 (first)	Level 2	[132362, 136457]
Q_2 (first)	Level 3	[136458, 140553]
Q_2 (second)	Level 3	[140554, 144649]
Q_1 (second)	Level 2	[144650, 148745]
Q_2 (third)	Level 3	[148746, 152841]
Q_2 (fourth)	Level 3	[152842, 156937]

IMPLEMENTATION

`snac_intercept.py` parses the model's flat output sequence by subtracting these offsets from the token IDs, grouping them into multiples of 7, and reconstructing the 3 hierarchical arrays required by the SNAC decoder.

Multi-Objective Reward Architecture

To ensure phonetic stability, voice preservation, and emotional compliance, the reward is calculated across three separate engines running on a dedicated evaluation GPU.



A Phonetic Accuracy Reward — R_{ASR}

Transcribes the generated waveform using `openai/whisper-large-v3` set to Hindi, then runs a strict Devanagari normalizer (`HindiNormalizer`) to expand numbers, dates, currencies, and nuktas — before computing Word Error Rate (WER) and Character Error Rate (CER).

$$R_{\text{WER}} = 1.0 - \tanh(3.0 \cdot \text{WER}) \quad R_{\text{CER}} = 1.0 - \tanh(3.0 \cdot \text{CER})$$

$$R_{\text{ASR}} = 0.7 \cdot R_{\text{WER}} + 0.3 \cdot R_{\text{CER}}$$

B Speaker Identity Reward — R_{spk}

Resamples generated audio to 16 kHz and extracts a speaker embedding using `speechbrain/spkrec-ecapa-voxceleb`, compared against the pre-extracted gold reference profile (`prakhar.wav` or `prerna.wav`) via cosine similarity:

$$\text{Sim} = \frac{\mathbf{e}_{\text{gen}} \cdot \mathbf{e}_{\text{gold}}}{\|\mathbf{e}_{\text{gen}}\| \|\mathbf{e}_{\text{gold}}\|}$$

Normalized to $[0.0, 1.0]$ over a typical target variance range:

$$R_{\text{spk}} = \text{clamp}\left(\frac{\text{Sim} - 0.2}{0.6}, 0.0, 1.0\right)$$

C Emotion / Style Reward — R_{emo}

Evaluates generated audio using the `superb/wav2vec2-base-superb-er` classifier. Manifest styles are mapped to classifier classes — `ANGER` → `angry`, `SURPRISE` → `surprise`, and `BOOK / CONV / WIKI / NEWS / BB / INDIC` → `neutral`. The reward is the classifier's probability of the target mapped category:

$$R_{\text{emo}} = \text{Prob}(\text{Target Category} \mid \text{Generated Waveform})$$

SECTION 05

Hardware Optimization Strategy

To prevent Out-Of-Memory failures under dual-T4 GPUs (2×15 GB, Kaggle), a split-GPU memory model separates policy training from reward evaluation.

CUDA:0 — POLICY & OPTIMIZER

≈ 7.5 GB

- Orpheus policy model, 3B parameters
- Loaded in fp16 LoRA ≈ 6.5 GB VRAM
- Handles forward / backward updates

CUDA:1 — REWARD EVALUATION

≈ 4 GB

- Whisper-large-v3 ≈ 3.1 GB
- ECAPA-TDNN ≈ 200 MB
- wav2vec2 Emotion Classifier ≈ 400 MB
- Models run sequentially

RESULT

Balanced VRAM profiles across both GPUs leave ample headroom for the parallel generation of the group size ($G = 4$) samples required by GRPO.